20.3 Discord 通道

> **生成模型**：Claude Opus 4.6 (anthropic/claude-opus-4-6)
> **Token 消耗**：输入 ~600k tokens，输出 ~55k tokens（本章合计）

Discord 是一个以社区和服务器（Guild）为核心的通信平台，在游戏、技术和教育领域广泛使用。OpenClaw 的 Discord 通道支持 DM（私聊）、服务器频道、线程等多种对话场景。

20.3.1 discord.js / Carbon 库的使用

OpenClaw 的 Discord 模块位于 `src/discord/` ，使用 discord.js 生态的客户端库连接 Discord Gateway（WebSocket API）。

核心架构：

```
Discord Gateway (WebSocket)
    │
    ▼
discord.js Client
    │
    ├── monitorDiscordProvider()     ← 消息监听入口
    │     ├── registerDiscordListener()
    │     └── createDiscordMessageHandler()
    │
    ├── send.ts                      ← 消息发送
    │     ├── send.messages.ts       ← 频道消息
    │     ├── send.guild.ts          ← 服务器消息
    │     └── send.outbound.ts       ← 出站投递
    │
    └── resolve-channels.ts          ← 频道解析
```

Discord 使用 **Snowflake ID** 作为所有实体（用户、消息、频道、服务器）的唯一标识——一个 64 位整数，编码了时间戳、工作节点 ID 和序列号。

> **衍生解释：Snowflake ID**
>
> Snowflake 是 Twitter（现 X）发明的分布式唯一 ID 生成算法。一个 Snowflake ID 包含：41 位时间戳（~69 年）+ 10 位机器 ID + 12 位序列号。Discord 采用了类似的方案，其 epoch 起始于 2015 年 1 月 1 日。这意味着 Snowflake ID 不仅是唯一的，还隐含了创建时间信息。

20.3.2 Guild 管理、DM 策略

Discord 的权限模型围绕 **Guild（服务器）** 展开。OpenClaw 的 Discord 配置支持精细的访问控制：

```
// src/discord/monitor/allow-list.ts（概念）
export function isDiscordGroupAllowedByPolicy(params): boolean {
  // 检查 Guild ID 是否在白名单中
  // 检查频道是否被允许
  // 检查发送者角色权限
}

export function resolveDiscordCommandAuthorized(params): boolean {
  // 命令级别的权限检查
}
```

DM（私聊）策略控制谁可以向 Bot 发送私信：

策略	说明
`allowFrom: ["*"]`	允许所有人
`allowFrom: ["12345"]`	仅允许指定用户 ID
不配置	默认拒绝所有 DM

Guild 消息的处理还涉及 **@提及检测**——在群组频道中，Bot 通常只在被 @提及时才响应：

```
// src/channels/dock.ts（Discord 部分）
mentions: {
  stripPatterns: () => ["<@!?\\d+>"],   // Discord 的提及格式
},
```

Discord 的提及格式为 `<@用户ID>` 或 `<@!用户ID>`，与 Telegram 的 `@username` 和 WhatsApp 的 `@电话号码` 都不同。

20.3.3 原生 Slash 命令与文本命令

Discord 支持**原生 Slash 命令**（Application Commands），用户在聊天框中输入 `/` 时会看到命令列表和参数提示。OpenClaw 通过 `createDiscordNativeCommand` 注册这些命令。

与 Telegram 的自定义命令类似，Discord 的 Slash 命令也需要注册到 Discord API，但实现方式更复杂——Discord 要求通过 Application Commands API 创建命令，并且支持子命令、选项类型、自动补全等高级功能。

OpenClaw 的 Discord 通道还支持普通文本命令（以配置的前缀开头），为不熟悉 Slash 命令的用户提供兼容入口。

20.3.4 每消息行数限制

Discord 客户端对长消息的渲染有特殊行为——超过一定行数的消息会被折叠。OpenClaw 的 Discord 分块器（`src/discord/chunk.ts`）同时考虑了**字符数**和**行数**两个维度：

```typescript
// src/discord/chunk.ts（简化）
const DEFAULT_MAX_CHARS = 2000;    // Discord 消息字符上限
const DEFAULT_MAX_LINES = 17;      // 软行数上限

export function chunkDiscordText(text, opts): string[] {
  const maxChars = opts.maxChars ?? DEFAULT_MAX_CHARS;
  const maxLines = opts.maxLines ?? DEFAULT_MAX_LINES;

  // 快速路径
  if (body.length <= maxChars && countLines(body) <= maxLines) {
    return [body];
  }

  // 逐行处理，同时追踪代码块（fence）状态
  let openFence: OpenFence | null = null;

  for (const line of lines) {
    // 检测代码块的开/关
    const fenceInfo = parseFenceLine(line);
    if (fenceInfo) {
      if (!openFence) openFence = fenceInfo;         // 打开代码块
      else if (fenceInfo.markerLen >= openFence.markerLen)
        openFence = null;                             // 关闭代码块
    }

    // 超过字符/行数限制时分块
    if ((wouldExceedChars || wouldExceedLines) && current.length > 0) {
      flush();    // flush 时自动关闭/重新打开代码块
    }
    // ...
  }
}
```

这个分块器最精妙的设计是**代码块平衡**：当分块点恰好在代码块内部时，它会在当前块末尾插入关闭标记（```），并在下一块开头重新打开代码块：

原始消息：

```python
def hello():
    print("hello")
def world():      ← 分块点在此
    print("world")
```

分块后：

块1: python def hello(): print("hello") ← 自动关闭

块2: python ← 自动重新打开 def world(): print("world")

此外还有一个 `rebalanceReasoningItalics` 函数，专门处理推理文本的斜体标记平衡——当推理文本（以 `_`

本节小结 {#本节小结 }

1. Discord 通道使用 discord.js 连接 Gateway WebSocket，支持 DM、频道、线程等多种对话场景。
2. **Guild 权限模型**支持服务器级、频道级和用户级的精细访问控制。
3. 支持**原生 Slash 命令**和**文本前缀命令**两种交互方式。
4. Discord 分块器同时考虑**字符数**（2000）和**行数**（17）两个维度，并能在分块时自动平衡代码块和斜体